C.E.S.A.R school

EBOOK DA DISCIPLINA · REVISÃO ENADE 2026

TEORIA DOS GRAFOS

Conceitos, algoritmos e aplicações explicados passo a passo, com questões no estilo ENADE, exercícios e respostas.

9 CAPÍTULOS · EXEMPLOS RESOLVIDOS · QUESTÕES COMENTADAS

Teoria dos Grafos · Revisão para o ENADE 2026
Prof. Laura Pacífico · CESAR School

Sumário

· **Apresentação** 3

1 Conceitos fundamentais 5

Origem: as pontes de Königsberg 5
Definições básicas 5
Grau e o lema do aperto de mãos 5
Famílias importantes 6
Subgrafos, complemento e isomorfismo 6
Passeios, trilhas, caminhos e ciclos 7
Conexidade 7
Exemplo passo a passo: a sequência de graus é realizável? 7

2 Representação computacional de grafos 10

Matriz de adjacência 10
Lista de adjacência 11
Matriz de incidência 11
Comparação 12
Exemplo passo a passo: contar passeios com A^2 12

3 Buscas em grafos e conexidade 14

Busca em largura (BFS) 14
Busca em profundidade (DFS) 15
BFS × DFS 16
Componentes conexas e bipartição 16
Ordenação topológica 16
Componentes fortemente conexas 17

4 Árvores e árvores geradoras mínimas 19

Caracterizações de árvore 19
Árvore geradora e AGM 19
Algoritmo de Kruskal 20
Algoritmo de Prim 20
Exemplo passo a passo 20

5 Caminhos mínimos 23

O problema e o relaxamento 23
Algoritmo de Dijkstra 23
Exemplo passo a passo: Dijkstra a partir de S 23
Pesos negativos e Bellman-Ford 24
Floyd-Warshall 25
Qual algoritmo usar 25

6 Grafos eulerianos e hamiltonianos 27

Grafos eulerianos 27
Exemplo passo a passo: a casinha 27
Problema do carteiro chinês 28
Grafos hamiltonianos 28
Problema do caixeiro viajante (PCV) 29
Euler × Hamilton e a fronteira entre P e NP 29

7 Planaridade 31

Definições 31
Fórmula de Euler e consequências 31
K_5, $K_{3,3}$ e o teorema de Kuratowski 32
Exemplos passo a passo 32
Grafo dual e mapas 32

8 Coloração de grafos 34

Definições e valores conhecidos 34
Algoritmo guloso e Welsh-Powell 34
Exemplo passo a passo: calendário de provas 35
Coloração de arestas e de mapas 35

9 Fluxos em redes e emparelhamentos 37

Redes de fluxo 37
Rede residual e caminho aumentante 37
Teorema do fluxo máximo e corte mínimo 37
Exemplo passo a passo: Edmonds-Karp 38
Emparelhamentos 38
Exemplo passo a passo: atribuição de monitores 39

A Quadro-resumo de algoritmos 40

B Respostas dos exercícios 41

C Referências 43

ANTES DE COMEÇAR

Apresentação

Este ebook cobre a disciplina de Teoria dos Grafos em nove capítulos, dos conceitos básicos aos fluxos em redes, e serve como material de estudo para o ENADE 2026.

Ele acompanha o site Revisão de Teoria dos Grafos · ENADE, os slides da disciplina e o livro-texto de Goldbarg e Goldbarg (2012). Onde os autores usam termos diferentes, o texto avisa.

Cada capítulo abre com a ideia central, desenvolve os conceitos em seções numeradas e resolve pelo menos um exemplo passo a passo, com cada etapa registrada em tabela ou figura. No fim, vêm os quadros de revisão e prática.

Os quadros que aparecem pelo caminho

QUADRO	O QUE TRAZ
A ideia central	O resumo do capítulo em duas ou três frases, logo na abertura.
Atenção	Armadilhas frequentes em questões objetivas.
Questão no estilo ENADE	Uma questão de múltipla escolha ou de asserção-razão, seguida do gabarito comentado.
Exercícios	Uma lista curta para praticar; as respostas estão no Apêndice B.

Tente resolver cada exercício antes de consultar a resposta: o raciocínio pedido nas provas é o mesmo.

Mapa dos capítulos

CAPÍTULO	TEMA	RESULTADOS E ALGORITMOS-CHAVE
1	Conceitos fundamentais	lema do aperto de mãos, famílias de grafos, isomorfismo, conexidade
2	Representação computacional	matriz de adjacência, lista de adjacência, matriz de incidência
3	Buscas e conexidade	BFS, DFS, componentes, bipartição, ordenação topológica
4	Árvores	caracterizações de árvore, Kruskal, Prim
5	Caminhos mínimos	Dijkstra, Bellman-Ford, Floyd-Warshall
6	Grafos eulerianos e hamiltonianos	teorema de Euler, Fleury, Dirac e Ore, caixeiro viajante
7	Planaridade	fórmula de Euler, K_5 e $K_{3,3}$, Kuratowski
8	Coloração	número cromático, algoritmo guloso, Welsh-Powell, quatro cores
9	Fluxos e emparelhamentos	Ford-Fulkerson, fluxo máximo e corte mínimo, Hall e König

O Apêndice A reúne a complexidade de todos os algoritmos em uma única tabela.

Notação

SÍMBOLO	SIGNIFICADO
G = (V, E)	grafo com conjunto de vértices V e conjunto de arestas E; há autores que escrevem G = (V, A)
n, m	número de vértices e número de arestas
uv	aresta entre u e v; em digrafos, (u, v) é o arco de u para v
N(v)	vizinhança de v: os vértices adjacentes a v
d(v)	grau de v; em digrafos, $d^+(v)$ é o grau de saída e $d^-(v)$, o de entrada
δ(G), Δ(G)	menor e maior grau de G
dist(u, v)	distância: comprimento do menor caminho de u até v
w(u, v)	peso (custo) da aresta ou do arco
χ(G), χ′(G)	número cromático e índice cromático

Pré-requisitos: conjuntos, relações, indução matemática e notação assintótica (O, Θ).

CAPÍTULO 1

1 CONCEITOS FUNDAMENTAIS

A IDEIA CENTRAL DO CAPÍTULO

Um grafo G = (V, E) é formado por um conjunto finito de vértices V e por um conjunto E de arestas, cada uma ligando dois vértices. Quase todo resultado da disciplina parte de três ideias deste capítulo: grau, caminho e conexidade.

1.1 Origem: as pontes de Königsberg

Em 1736, Euler analisou se era possível passear por Königsberg cruzando cada uma das sete pontes exatamente uma vez. Ele trocou as quatro regiões de terra por vértices e as pontes por arestas. Os graus ficaram 5, 3, 3 e 3; com quatro vértices de grau ímpar, o passeio é impossível, como mostra o Capítulo 6.

1.2 Definições básicas

- **Adjacência e incidência:** u e v são adjacentes (vizinhos) se existe a aresta uv; essa aresta é incidente a u e a v.
- **Laço:** aresta que liga um vértice a ele mesmo. **Arestas paralelas** (ou múltiplas): duas ou mais arestas com as mesmas extremidades.
- **Grafo simples:** não tem laços nem arestas paralelas. **Multigrafo:** admite arestas paralelas e, conforme o autor, laços.
- **Digrafo** (grafo direcionado): cada arco (u, v) tem origem u e destino v.
- **Grafo ponderado** (valorado): cada aresta tem um peso w(u, v), como distância, custo ou capacidade.
- **Ordem e tamanho:** a ordem é o número de vértices, n; o tamanho é o número de arestas, m.

Salvo aviso, “grafo” neste ebook significa grafo simples, finito e não direcionado.

1.3 Grau e o lema do aperto de mãos

O grau d(v) é o número de arestas incidentes a v, e um laço conta 2. Como cada aresta soma 2 ao total dos graus, vale o lema do aperto de mãos:

$$\sum_{v \in V} d(v) = 2m$$

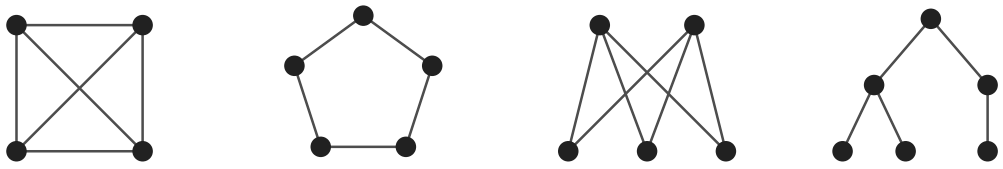
Consequências diretas:

- O número de vértices de grau ímpar é sempre par.
- Em um grafo k-regular (todos os vértices com grau k), $m = n \cdot k/2$; por isso $n \cdot k$ precisa ser par.
- Em digrafos, a soma dos graus de saída e a soma dos graus de entrada valem m cada uma.
- Vértice isolado tem grau 0; vértice pendente (folha) tem grau 1.

1.4 Famílias importantes

FAMÍLIA	DEFINIÇÃO	NÚMERO DE ARESTAS
Completo K_n	todo par de vértices é adjacente	$n(n-1)/2$
Ciclo C_n, $n \geq 3$	vértices em anel, todos com grau 2	n
Caminho P_n	n vértices em linha	$n-1$
Bipartido completo $K_{r,s}$	partes com r e s vértices; cada vértice de uma parte liga-se a todos da outra	$r \cdot s$
Hipercubo Q_k	2^k vértices (palavras binárias de k bits), adjacentes quando diferem em um bit	$k \cdot 2^{k-1}$
Árvore	conexo e sem ciclos	$n-1$

Um grafo é bipartido quando V se divide em duas partes X e Y e toda aresta liga X a Y. Teorema: um grafo é bipartido se e somente se não tem ciclo de comprimento ímpar.



K_4
completo: m = 6

C_5
ciclo: m = 5

$K_{2,3}$
bipartido completo: m = 2·3 = 6

Árvore
n = 6, m = n − 1 = 5

Figura 1.1 · As fórmulas da tabela aplicadas da esquerda para a direita: K_4 tem $4 \cdot 3/2 = 6$ arestas, C_5 tem 5, $K_{2,3}$ tem $2 \cdot 3 = 6$, e a árvore de 6 vértices tem 5.

1.5 Subgrafos, complemento e isomorfismo

- **Subgrafo H de G:** os vértices e as arestas de H estão em G.
- **Subgrafo gerador** (abrangente): usa todos os vértices de G e parte das arestas.
- **Subgrafo induzido G[S]:** escolhe os vértices de S e leva todas as arestas de G entre eles.
- **Complemento $\bar{G}$:** mesmos vértices; uv é aresta de $\bar{G}$ se e somente se não é aresta de G. Assim, $m(G) + m(\bar{G}) = n(n-1)/2$.
- **Clique:** vértices dois a dois adjacentes. **Conjunto independente:** vértices dois a dois não adjacentes.

G e H são isomorfos quando existe uma bijeção f entre seus vértices tal que uv é aresta de G se e somente se f(u)f(v) é aresta de H. Para provar que dois grafos não são isomorfos, basta achar um invariante diferente: n, m, sequência de graus, número de triângulos, conexidade ou bipartição.

Invariantes iguais não provam isomorfismo. C_6 e dois triângulos disjuntos têm seis vértices de grau 2, mas só o primeiro é conexo.

1.6 Passeios, trilhas, caminhos e ciclos

A terminologia varia entre autores, então confira a definição usada no enunciado. Alguns livros brasileiros chamam de cadeia o percurso em grafo não direcionado e reservam caminho para digrafos.

TERMO	REPETE VÉRTICE?	REPETE ARESTA?	DEFINIÇÃO
Passeio	sim	sim	sequência de vértices em que consecutivos são adjacentes
Trilha	sim	não	passeio sem aresta repetida; se fechada, é um circuito
Caminho	não	não	passeio sem vértice repetido
Ciclo	só o inicial, no fim	não	caminho fechado com pelo menos 3 arestas

O comprimento de um percurso é o número de arestas. A distância dist(u, v) é o comprimento do menor caminho entre u e v, e o diâmetro é a maior distância entre dois vértices do grafo.

1.7 Conexidade

- **Conexo:** existe caminho entre todo par de vértices. **Componente conexa** é um subgrafo conexo maximal.
- **Ponte:** aresta cuja remoção aumenta o número de componentes. **Vértice de corte** (articulação): vértice com a mesma propriedade.
- **Digrafo fortemente conexo:** há caminho de u para v e de v para u, para todo par. **Fracamente conexo:** o grafo sem as direções é conexo.
- Todo grafo conexo tem $m \geq n - 1$.
- Todo grafo simples com mais de $(n - 1)(n - 2)/2$ arestas é conexo, pois o grafo desconexo com mais arestas possíveis é K_{n-1} mais um vértice isolado.

1.8 Exemplo passo a passo: a sequência de graus é realizável?

Uma sequência de inteiros é gráfica quando existe um grafo simples com exatamente aqueles graus. O teste de Havel-Hakimi ordena a sequência de forma decrescente, retira o maior valor d_1 e subtrai 1 dos d_1 valores seguintes. Repete-se até sobrarem só zeros (gráfica) ou surgir um valor negativo (não gráfica).

Sequência (3, 3, 2, 2, 2): a soma é 12, então o grafo teria m = 6.

PASSO	SEQUÊNCIA	AÇÃO
1	3, 3, 2, 2, 2	retira 3 e subtrai 1 dos três seguintes
2	2, 1, 1, 2, reordenada para 2, 2, 1, 1	retira 2 e subtrai 1 dos dois seguintes
3	1, 0, 1, reordenada para 1, 1, 0	retira 1 e subtrai 1 do seguinte
4	0, 0	só zeros: a sequência é gráfica

Sequência (3, 3, 3, 1): a soma é 10, par, mas o teste falha.

PASSO	SEQUÊNCIA	AÇÃO
1	3, 3, 3, 1	retira 3 e subtrai 1 dos três seguintes
2	2, 2, 0	retira 2 e subtrai 1 dos dois seguintes
3	1, −1	valor negativo: não é gráfica

A intuição confirma: com 4 vértices, quem tem grau 3 é vizinho de todos, e três vértices assim dariam grau 3 também ao último. Soma par é condição necessária, mas não suficiente.

! ATENÇÃO

- Um laço soma 2 ao grau, não 1.
- Mesma sequência de graus não garante isomorfismo; um invariante diferente garante que não há isomorfismo.
- Subgrafo induzido leva todas as arestas entre os vértices escolhidos; um subgrafo qualquer pode omitir algumas.
- "n vértices e n − 1 arestas" só implica árvore se o grafo for conexo ou acíclico.

Q QUESTÃO NO ESTILO ENADE

Considere as afirmações sobre grafos simples.

I. Todo grafo com n vértices e n − 1 arestas é uma árvore.

II. Em qualquer grafo, o número de vértices de grau ímpar é par.

III. O grafo $K_{3,3}$ tem 9 arestas e é bipartido.

IV. Dois grafos com a mesma sequência de graus são isomorfos.

É correto apenas o que se afirma em:

(A) I e II.

(B) II e III.

(C) III e IV.

(D) I, II e III.

(E) II, III e IV.

✓ GABARITO COMENTADO

Gabarito: B.

- I é falsa: um triângulo mais um vértice isolado tem 4 vértices e 3 arestas, mas não é conexo.
- II é verdadeira, pelo lema do aperto de mãos.
- III é verdadeira: são 3 × 3 = 9 arestas, e as duas partes de 3 vértices formam a bipartição.
- IV é falsa: C_6 e dois triângulos disjuntos têm a mesma sequência de graus.

Exercícios

1. Quantas arestas tem K_{10}?
2. Um grafo 3-regular tem 12 arestas. Quantos vértices ele tem?
3. Existe grafo 3-regular com 9 vértices? Justifique.
4. Mostre que C_5 é isomorfo ao seu complemento.
5. Um grafo simples com 8 vértices e 22 arestas é necessariamente conexo?

CAPÍTULO 2

2 REPRESENTAÇÃO COMPUTACIONAL DE GRAFOS

A IDEIA CENTRAL DO CAPÍTULO

A estrutura de dados define o custo de cada operação: a matriz de adjacência diz se u e v são vizinhos em $O(1)$ e ocupa $\Theta(n^2)$ de memória; a lista de adjacência ocupa $\Theta(n + m)$ e percorre os vizinhos de v em $O(d(v))$.

O exemplo do capítulo é o grafo G com V = {1, 2, 3, 4, 5} e E = {12, 13, 23, 24, 35, 45}, reaproveitado no Capítulo 3.

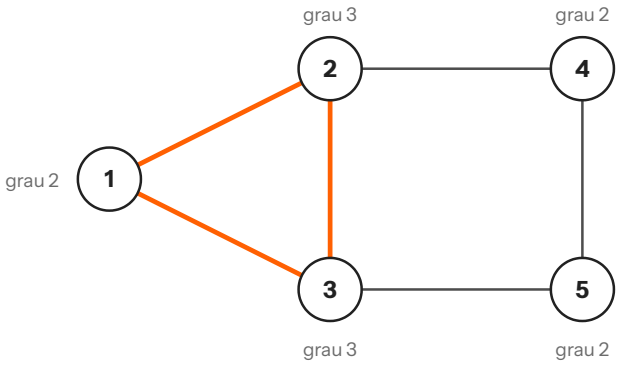


Figura 2.1 · Grafo G, com V = {1, 2, 3, 4, 5} e E = {12, 13, 23, 24, 35, 45}. A soma dos graus é 2 + 3 + 3 + 2 + 2 = 12 = 2m; o triângulo destacado é o ciclo ímpar que impede G de ser bipartido (Capítulo 3).

2.1 Matriz de adjacência

A matriz A, de ordem n × n, tem A[i][j] = 1 se ij é aresta e 0 caso contrário. Em grafos ponderados, guarda-se o peso, e a ausência de aresta vira 0 ou ∞, conforme o problema.

- Em grafos não direcionados, A é simétrica; em grafos simples, a diagonal é nula.
- A soma da linha i é o grau de i. Em digrafos, a linha dá o grau de saída e a coluna, o de entrada.
- O elemento (i, j) de A^k conta os passeios de comprimento k entre i e j.

A	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	1	1	0	0	1
4	0	1	0	0	1
5	0	0	1	1	0

2.2 Lista de adjacência

É um vetor de n listas, em que a lista de v guarda os vizinhos de v. Em grafo não direcionado, cada aresta aparece duas vezes, então há 2m entradas no total (12, no exemplo).

```
1: 2, 3
2: 1, 3, 4
3: 1, 2, 5
4: 2, 5
5: 3, 4
```

Em Python, um dicionário de listas é a forma mais direta:

```
G = {1: [2, 3], 2: [1, 3, 4], 3: [1, 2, 5], 4: [2, 5], 5: [3, 4]}
grau = {v: len(viz) for v, viz in G.items()}   # {1: 2, 2: 3, 3: 3, 4: 2, 5: 2}
```

2.3 Matriz de incidência

A matriz B, de ordem n × m, tem B[v][e] = 1 se v é extremidade da aresta e. Cada coluna tem exatamente dois 1s, e a soma da linha v é d(v). Em digrafos, usa-se −1 na origem e +1 no destino; alguns autores invertem os sinais.

B	e1 = 12	e2 = 13	e3 = 23	e4 = 24	e5 = 35	e6 = 45
1	1	1	0	0	0	0
2	1	0	1	1	0	0
3	0	1	1	0	1	0
4	0	0	0	1	0	1
5	0	0	0	0	1	1

A matriz de incidência ocupa $\Theta(n \cdot m)$ e aparece mais em modelos de otimização, como fluxos em redes, do que na implementação de buscas.

2.4 Comparação

CRITÉRIO	MATRIZ DE ADJACÊNCIA	LISTA DE ADJACÊNCIA
Memória	$\Theta(n^2)$	$\Theta(n + m)$
Testar se uv é aresta	$O(1)$	$O(d(u))$
Percorrer os vizinhos de v	$\Theta(n)$	$O(d(v))$
BFS ou DFS completa	$\Theta(n^2)$	$\Theta(n + m)$
Melhor uso	grafos densos, com m próximo de n^2	grafos esparsos, com m muito menor que n^2

2.5 Exemplo passo a passo: contar passeios com A^2

Para $i \neq j$, o elemento (i, j) de A^2 é o número de vizinhos comuns de i e j, ou seja, de passeios i–k–j. Na diagonal, o elemento (i, i) é o grau de i.

A^2	1	2	3	4	5
1	2	1	1	1	1
2	1	3	1	0	2
3	1	1	3	2	0
4	1	0	2	2	0
5	1	2	0	0	2

1. **Elemento (1, 4):** N(1) = {2, 3} e N(4) = {2, 5} têm em comum só o 2, logo há 1 passeio (1–2–4).
2. **Elemento (2, 5):** N(2) = {1, 3, 4} e N(5) = {3, 4} têm em comum 3 e 4, logo há 2 passeios (2–3–5 e 2–4–5).
3. **Elemento (2, 4):** N(2) e N(4) não têm vizinho comum, logo vale 0, embora 2 e 4 sejam adjacentes.
4. **Diagonal (1, 1) = 2 = d(1):** são os passeios 1–2–1 e 1–3–1.

Vale também: o traço de A^3 dividido por 6 é o número de triângulos. Em G, o traço de A^3 é 6, e o único triângulo é 1–2–3.

ATENÇÃO

- Se a questão mostra uma matriz de adjacência não simétrica, o grafo é direcionado.
- Lista de adjacência de grafo não direcionado tem 2m entradas, não m.
- Percorrer o grafo em O(n + m) vale para a lista; com matriz, o custo é $\Theta(n^2)$ mesmo com poucas arestas.

QUESTÃO NO ESTILO ENADE

Uma rede social tem 1 milhão de usuários (vértices), e cada um tem cerca de 100 amizades (arestas não direcionadas). Sobre a representação desse grafo em memória, é correto afirmar:

(A) A matriz de adjacência é preferível, pois ocupa memória proporcional ao número de arestas.

(B) A lista de adjacência é preferível, pois ocupa $\Theta(n + m)$ e o grafo é esparso.

(C) As duas estruturas ocupam a mesma memória, pois o grafo não é direcionado.

(D) A matriz de incidência é a mais compacta, pois tem uma coluna por vértice.

(E) A lista de adjacência impede verificar se dois usuários são amigos.

GABARITO COMENTADO

Gabarito: B. Com $n = 10^6$ e $m \approx 5 \times 10^7$ (cada amizade conta para dois usuários), a matriz teria 10^{12} posições, e a lista guarda cerca de 10^8 entradas.

- A erra a memória da matriz, que é $\Theta(n^2)$.
- C ignora a diferença de quatro ordens de grandeza.
- D erra a dimensão da matriz de incidência, que é $n \times m$.
- E é falsa: basta percorrer a lista de um dos usuários, em $O(d(u))$.

Exercícios

1. Monte a matriz de adjacência do digrafo com arcos (1, 2), (2, 3), (3, 1) e (3, 4). Quais são os graus de entrada e de saída do vértice 3?
2. A matriz de adjacência de um grafo simples tem 20 elementos iguais a 1. Quantas arestas tem o grafo?
3. Pela tabela de A^2, quantos passeios de comprimento 2 existem entre 3 e 4? Quais são?
4. Qual estrutura você usaria para K_{1000}? Por quê?

CAPÍTULO 3

3 BUSCAS EM GRAFOS E CONEXIDADE

A IDEIA CENTRAL DO CAPÍTULO

BFS e DFS visitam todos os vértices e arestas em O(n + m) com lista de adjacência. A BFS entrega as distâncias, em número de arestas, a partir da origem; a DFS revela a estrutura do grafo: ciclos, ordem topológica e componentes.

Os exemplos usam o grafo G da Figura 2.1, percorrendo os vizinhos em ordem crescente.

3.1 Busca em largura (BFS)

A BFS usa uma fila: visita a origem, depois todos os vizinhos dela, depois os vizinhos dos vizinhos, camada por camada.

```
BFS(G, s)
  para cada v em V: visitado[v] ← falso; dist[v] ← ∞; pai[v] ← nulo
  visitado[s] ← verdadeiro; dist[s] ← 0
  Q ← fila vazia; ENFILEIRA(Q, s)
  enquanto Q não estiver vazia:
      u ← DESENFILEIRA(Q)
      para cada v em Adj[u]:
          se não visitado[v]:
              visitado[v] ← verdadeiro
              dist[v] ← dist[u] + 1; pai[v] ← u
              ENFILEIRA(Q, v)
```

Exemplo passo a passo: BFS a partir do vértice 1.

PASSO	SAI DA FILA	DESCOBERTOS: DIST (PAI)	FILA APÓS O PASSO
0	—	1: 0 (—)	1
1	1	2: 1 (1); 3: 1 (1)	2, 3
2	2	4: 2 (2)	3, 4
3	3	5: 2 (3)	4, 5
4	4	nenhum	5
5	5	nenhum	vazia

A ordem de visita é 1, 2, 3, 4, 5, com camadas {1}, {2, 3} e {4, 5}. A árvore de BFS tem as arestas 12, 13, 24 e 35; nela, o caminho de 1 até 5 (1–3–5) é mínimo em número de arestas.

3.2 Busca em profundidade (DFS)

A DFS avança enquanto encontra vizinho não visitado e recua quando não há para onde ir; usa uma pilha, explícita ou pela recursão. Ela registra o tempo de descoberta d[v] e o de término f[v] de cada vértice.

```
DFS(G)
  para cada v em V: visitado[v] ← falso
  tempo ← 0
  para cada v em V:
      se não visitado[v]: VISITA(v)

VISITA(u)
  tempo ← tempo + 1; d[u] ← tempo; visitado[u] ← verdadeiro
  para cada v em Adj[u]:
      se não visitado[v]:
          pai[v] ← u; VISITA(v)
  tempo ← tempo + 1; f[u] ← tempo
```

Exemplo passo a passo: DFS a partir do vértice 1.

VÉRTICE	DESCOBERTA D	TÉRMINO F	PAI
1	1	10	—
2	2	9	1
3	3	8	2
5	4	7	3
4	5	6	5

A busca seguiu 1 → 2 → 3 → 5 → 4. Em 4, os vizinhos 2 e 5 já estavam visitados, e a busca recuou até esvaziar a pilha.

- **Arestas de árvore:** 12, 23, 35 e 54.
- **Arestas de retorno:** 13 e 24. Cada uma fecha um ciclo: 1–2–3 e 2–3–5–4.

Propriedades:

- **Teorema dos parênteses:** os intervalos [d, f] de dois vértices são disjuntos ou um contém o outro.
- Um grafo tem ciclo se e somente se a DFS encontra aresta de retorno.
- Em digrafos, as arestas se classificam em árvore, retorno, avanço e cruzamento. Em grafos não direcionados, só existem as duas primeiras.

3.3 BFS × DFS

CRITÉRIO	BFS	DFS
Estrutura auxiliar	fila (FIFO)	pilha (LIFO) ou recursão
Custo com lista de adjacência	$O(n + m)$	$O(n + m)$
Custo com matriz de adjacência	$O(n^2)$	$O(n^2)$
O que entrega	distâncias em número de arestas, camadas	tempos d e f, arestas de retorno
Aplicações típicas	menor caminho sem pesos, teste de bipartição	detecção de ciclos, ordenação topológica, componentes fortemente conexas

3.4 Componentes conexas e bipartição

- **Componentes:** rode a busca a partir de cada vértice ainda não visitado; cada rodada encontra uma componente, e o total continua O(n + m).
- **Bipartição:** pinte as camadas pares da BFS de uma cor e as ímpares de outra. O grafo é bipartido se e somente se nenhuma aresta liga dois vértices da mesma cor.
- Em G, os vértices 2 e 3 estão na camada 1 e são vizinhos, então G não é bipartido: o triângulo 1–2–3 é um ciclo ímpar.

3.5 Ordenação topológica

Uma ordenação topológica de um digrafo lista os vértices de modo que todo arco (u, v) tenha u antes de v. Ela existe se e somente se o digrafo é acíclico (DAG, do inglês *directed acyclic graph*). Há dois métodos, ambos O(n + m):

- **Kahn:** retire repetidamente um vértice com grau de entrada zero e apague seus arcos.
- **DFS:** liste os vértices em ordem decrescente de tempo de término f.

Exemplo passo a passo (Kahn): uma grade curricular com seis disciplinas. Os arcos são Programação → Estruturas de Dados, Estruturas de Dados → Grafos, Estruturas de Dados → Banco de Dados, Matemática Discreta → Grafos, Grafos → IA e Matemática Discreta → IA. Empates são decididos em ordem alfabética.

PASSO	DISPONÍVEIS (GRAU DE ENTRADA 0)	RETIRADO	EFEITO
1	Matemática Discreta, Programação	Matemática Discreta	Grafos e IA perdem um pré-requisito
2	Programação	Programação	Estruturas de Dados fica livre
3	Estruturas de Dados	Estruturas de Dados	Banco de Dados e Grafos ficam livres
4	Banco de Dados, Grafos	Banco de Dados	nenhum
5	Grafos	Grafos	IA fica livre
6	IA	IA	fim

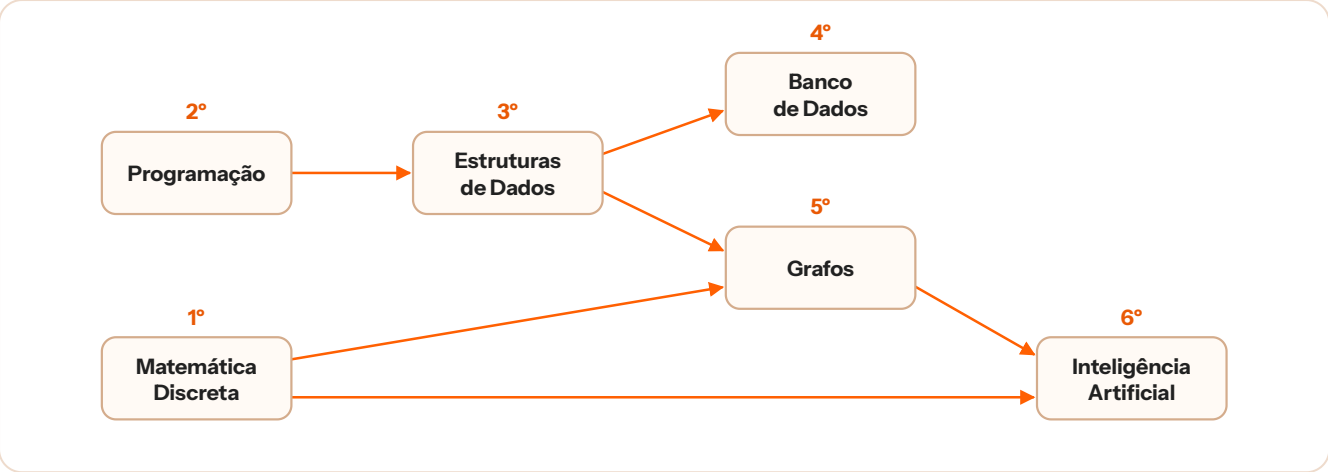


Figura 3.1 · Pré-requisitos da grade; de 1º a 6º, a ordem encontrada por Kahn. Toda seta vai de uma disciplina para outra de número maior, que é exatamente a condição de uma ordenação topológica.

Se o algoritmo parar com vértices restantes e nenhum deles tiver grau de entrada zero, o digrafo tem ciclo e não há ordenação topológica.

3.6 Componentes fortemente conexas

Em um digrafo, uma componente fortemente conexa é um conjunto maximal de vértices que se alcançam mutuamente. O algoritmo de Kosaraju resolve o problema em $O(n + m)$:

1. Rode a DFS em G e anote os tempos de término.
2. Construa o grafo transposto G^T, com todos os arcos invertidos.
3. Rode a DFS em G^T, iniciando pelos vértices em ordem decrescente de término; cada árvore obtida é uma componente.

O algoritmo de Tarjan chega ao mesmo resultado com uma única DFS. Contraindo cada componente em um vértice, obtém-se um DAG.

ATENÇÃO

- A BFS só garante caminhos mínimos quando todas as arestas têm o mesmo peso.
- A ordem de visita depende da ordem das listas de adjacência; questões costumam fixar ordem crescente ou alfabética.
- A ordenação topológica não é única: a grade acima também aceita Programação, Matemática Discreta, Estruturas de Dados, Grafos, Banco de Dados, IA.
- Ordenação topológica só existe em digrafos acíclicos.

QUESTÃO NO ESTILO ENADE

Uma rede social é modelada como grafo não direcionado: vértices são usuários e arestas são amizades. Deseja-se calcular, para um usuário s, o menor número de intermediários até cada outro usuário. Qual abordagem resolve o problema corretamente e com a menor complexidade assintótica?

(A) Busca em profundidade a partir de s, em $O(n + m)$.

(B) Busca em largura a partir de s, em $O(n + m)$.

(C) Algoritmo de Dijkstra a partir de s, em $O((n + m) \log n)$.

(D) Algoritmo de Floyd-Warshall, em $O(n^3)$.

(E) Ordenação topológica do grafo, em $O(n + m)$.

GABARITO COMENTADO

Gabarito: B. Com arestas sem peso, a camada da BFS é exatamente a distância.

- A: a DFS não garante o menor número de intermediários.
- C: Dijkstra acerta, mas custa mais que a BFS quando todos os pesos são iguais.
- D: calcula todos os pares, com custo desnecessário.
- E: ordenação topológica exige digrafo acíclico.

Exercícios

1. Execute a BFS a partir do vértice 4 em G. Dê a ordem de visita e as distâncias.
2. Execute a DFS a partir do vértice 5 em G e classifique as arestas.
3. Por que a BFS com matriz de adjacência custa $\Theta(n^2)$?
4. Encontre, pela DFS, uma ordem topológica para a grade do exemplo, visitando as disciplinas em ordem alfabética.

CAPÍTULO 4

4 ÁRVORES E ÁRVORES GERADORAS MÍNIMAS

A IDEIA CENTRAL DO CAPÍTULO

Uma árvore é um grafo conexo e sem ciclos; com n vértices, tem exatamente n − 1 arestas. A árvore geradora mínima (AGM) de um grafo ponderado conexo liga todos os vértices com o menor custo total e é obtida pelos algoritmos gulosos de Kruskal e Prim.

4.1 Caracterizações de árvore

Para um grafo T com n vértices, as afirmações abaixo são equivalentes; qualquer uma serve como definição.

1. T é conexo e acíclico.
2. T é conexo e tem n − 1 arestas.
3. T é acíclico e tem n − 1 arestas.
4. Existe exatamente um caminho entre cada par de vértices.
5. T é conexo, e toda aresta é ponte.
6. T é acíclico, e acrescentar qualquer aresta cria exatamente um ciclo.

Fatos que caem em prova:

- Toda árvore com $n \geq 2$ vértices tem pelo menos duas folhas.
- **Floresta** é um grafo acíclico; com k componentes, tem $m = n - k$ arestas.
- **Fórmula de Cayley:** há n^{n-2} árvores distintas com n vértices rotulados, que é também o número de árvores geradoras de K_n.
- **Árvore enraizada:** a escolha de uma raiz define pai, filhos, nível e altura. Uma árvore binária de altura h tem no máximo $2^{h+1} - 1$ nós.

4.2 Árvore geradora e AGM

Uma árvore geradora de G é um subgrafo gerador que é árvore. Todo grafo conexo tem uma, e as árvores da BFS e da DFS são exemplos. Em grafo ponderado, a AGM é a árvore geradora com a menor soma de pesos.

Duas propriedades sustentam os algoritmos:

- **Propriedade do corte:** para qualquer divisão dos vértices em dois grupos, a aresta mais leve que cruza a divisão pertence a alguma AGM, e a toda AGM se os pesos forem distintos.

- **Propriedade do ciclo:** a aresta mais pesada de um ciclo, se for única, não pertence a nenhuma AGM.

Com pesos distintos, a AGM é única.

4.3 Algoritmo de Kruskal

Kruskal ordena as arestas por peso e aceita cada uma que não forme ciclo com as já escolhidas.

```
KRUSKAL(G, w)
  T ← ∅
  cada vértice começa em seu próprio conjunto
  para cada aresta uv, em ordem crescente de peso:
      se u e v estão em conjuntos diferentes:
          T ← T ∪ {uv}; UNE(u, v)
  devolva T
```

O custo é O(m log m), dominado pela ordenação, com a estrutura union-find para testar ciclos. Em grafo desconexo, Kruskal devolve uma floresta geradora mínima.

4.4 Algoritmo de Prim

Prim cresce uma única árvore a partir de uma raiz r, sempre pela aresta mais leve que sai da árvore.

```
PRIM(G, w, r)
  para cada v em V: chave[v] ← ∞; pai[v] ← nulo
  chave[r] ← 0; Q ← V   (fila de prioridade por chave)
  enquanto Q não estiver vazia:
      u ← EXTRAI-MÍNIMO(Q)
      para cada v em Adj[u]:
          se v está em Q e w(u, v) < chave[v]:
              chave[v] ← w(u, v); pai[v] ← u
```

O custo é O(m log n) com heap binário e $O(n^2)$ com vetor, versão melhor para grafos densos.

4.5 Exemplo passo a passo

O grafo tem 6 vértices e 9 arestas: AB 4, AC 2, BC 5, BD 7, CD 6, CE 8, DE 3, DF 10 e EF 9.

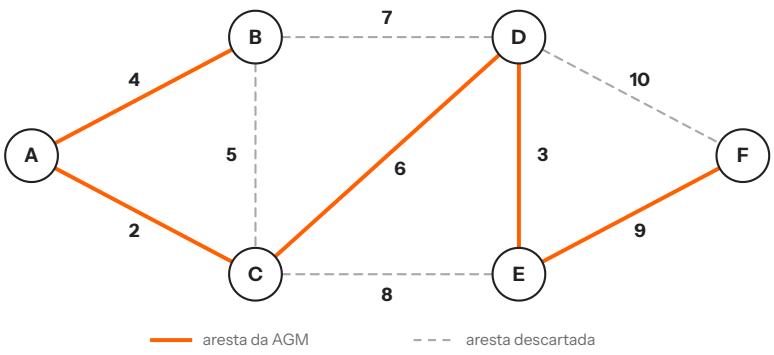


Figura 4.1 · A AGM usa 5 das 9 arestas e custa 2 + 3 + 4 + 6 + 9 = 24. Cada aresta tracejada fecharia um ciclo formado por arestas mais leves; na árvore, o caminho A–C–D–E custa 11, mais que A–C–E no grafo.

Kruskal examina as arestas em ordem crescente de peso:

ORDEM	ARESTA	PESO	DECISÃO	COMPONENTES DEPOIS
1	AC	2	aceita	{A, C} {B} {D} {E} {F}
2	DE	3	aceita	{A, C} {B} {D, E} {F}
3	AB	4	aceita	{A, B, C} {D, E} {F}
4	BC	5	rejeita: fecharia o ciclo A–B–C	sem mudança
5	CD	6	aceita	{A, B, C, D, E} {F}
6	BD	7	rejeita: B e D já estão ligados	sem mudança
7	CE	8	rejeita: C e E já estão ligados	sem mudança
8	EF	9	aceita	{A, B, C, D, E, F}

Com n − 1 = 5 arestas, o algoritmo para, e DF (10) nem é examinada.

Prim a partir de A escolhe, a cada passo, a aresta mais leve entre a árvore e o resto:

PASSO	ÁRVORE ATUAL	CANDIDATAS (PESO)	ESCOLHIDA
1	A	AB 4, AC 2	AC 2
2	A, C	AB 4, BC 5, CD 6, CE 8	AB 4
3	A, B, C	BD 7, CD 6, CE 8	CD 6
4	A, B, C, D	CE 8, DE 3, DF 10	DE 3
5	A, B, C, D, E	DF 10, EF 9	EF 9

Os dois chegam à mesma AGM, {AC, AB, CD, DE, EF}, de custo 2 + 4 + 6 + 3 + 9 = 24; com pesos distintos, ela é única.

ATENÇÃO

- AGM não é árvore de caminhos mínimos. Na AGM do exemplo, o caminho de A a E é A–C–D–E, de custo 11, mas no grafo A–C–E custa 10.
- Kruskal escolhe arestas no grafo todo; Prim mantém uma única árvore conectada.
- Pesos negativos não atrapalham Kruskal nem Prim.
- Com pesos repetidos, pode haver várias AGMs, todas com o mesmo custo.

QUESTÃO NO ESTILO ENADE

Considere um grafo G conexo, não direcionado e ponderado, com pesos distintos, e as afirmações:

I. A árvore geradora mínima de G é única.

II. A aresta de menor peso de G pertence à árvore geradora mínima.

III. A aresta de maior peso de G nunca pertence à árvore geradora mínima.

IV. O caminho entre dois vértices na árvore geradora mínima é o caminho de menor custo entre eles em G.

É correto apenas o que se afirma em:

(A) I e II.

(B) I e III.

(C) II e IV.

(D) I, II e III.

(E) I, II, III e IV.

GABARITO COMENTADO

Gabarito: A.

- I é verdadeira: pesos distintos garantem AGM única.
- II é verdadeira: isolando uma das extremidades da aresta mais leve, ela é a mais leve a cruzar esse corte.
- III é falsa: se a aresta mais pesada for ponte, toda árvore geradora precisa dela.
- IV é falsa, como mostra o caminho de A a E no exemplo.

Exercícios

1. Uma floresta tem 15 vértices e 11 arestas. Quantas árvores a compõem?
2. Execute Prim a partir de F no grafo do exemplo.
3. Quantas árvores geradoras tem K_4?
4. Se o peso de CD passar de 6 para 11, qual será a nova AGM e seu custo?
5. Uma árvore tem 3 vértices de grau 3, 2 vértices de grau 2 e os demais de grau 1. Quantas folhas ela tem?

CAPÍTULO 5

5 CAMINHOS MÍNIMOS

A IDEIA CENTRAL DO CAPÍTULO

Dijkstra resolve caminhos mínimos a partir de uma origem quando todos os pesos são não negativos; Bellman-Ford aceita pesos negativos e detecta ciclos negativos; Floyd-Warshall calcula as distâncias entre todos os pares em $O(n^3)$.

5.1 O problema e o relaxamento

Em um grafo ponderado, o custo de um caminho é a soma dos pesos de seus arcos, e dist(s, v) é o menor custo de s até v. Todos os algoritmos do capítulo repetem a operação de relaxamento:

```
RELAXA(u, v)
  se dist[u] + w(u, v) < dist[v]:
      dist[v] ← dist[u] + w(u, v); pai[v] ← u
```

Subcaminhos de caminhos mínimos também são mínimos (subestrutura ótima). Se houver ciclo de custo negativo alcançável, o caminho mínimo não está definido, pois cada volta no ciclo reduz o custo.

5.2 Algoritmo de Dijkstra

Dijkstra é guloso: extrai sempre o vértice de menor distância provisória e passa a considerá-la definitiva.

```
DIJKSTRA(G, w, s)
  para cada v em V: dist[v] ← ∞; pai[v] ← nulo
  dist[s] ← 0
  Q ← V   (fila de prioridade por dist)
  enquanto Q não estiver vazia:
      u ← EXTRAI-MÍNIMO(Q)
      para cada v em Adj[u]: RELAXA(u, v)
```

O custo é $O(n^2)$ com vetor e $O((n + m) \log n)$ com heap binário. A correção depende de todos os pesos serem não negativos.

5.3 Exemplo passo a passo: Dijkstra a partir de S

Os arcos são S→A 7, S→B 2, B→A 3, B→D 8, A→C 4, A→D 2, C→E 3 e D→E 1. S começa com distância 0. Na tabela, “5 / B” significa distância provisória 5 com predecessor B, e ✓ marca vértice já definitivo.

ITERAÇÃO	EXTRAÍDO (DIST)	A	B	C	D	E
início	—	∞	∞	∞	∞	∞
1	S (0)	7 / S	2 / S	∞	∞	∞
2	B (2)	5 / B	✓	∞	10 / B	∞
3	A (5)	✓	✓	9 / A	7 / A	∞
4	D (7)	✓	✓	9 / A	✓	8 / D
5	E (8)	✓	✓	9 / A	✓	✓
6	C (9)	✓	✓	✓	✓	✓

Resultado: dist(B) = 2, dist(A) = 5, dist(D) = 7, dist(E) = 8 e dist(C) = 9. Seguindo os predecessores a partir de E, o caminho é S → B → A → D → E. Duas distâncias melhoraram no percurso: A caiu de 7 para 5, e D, de 10 para 7.

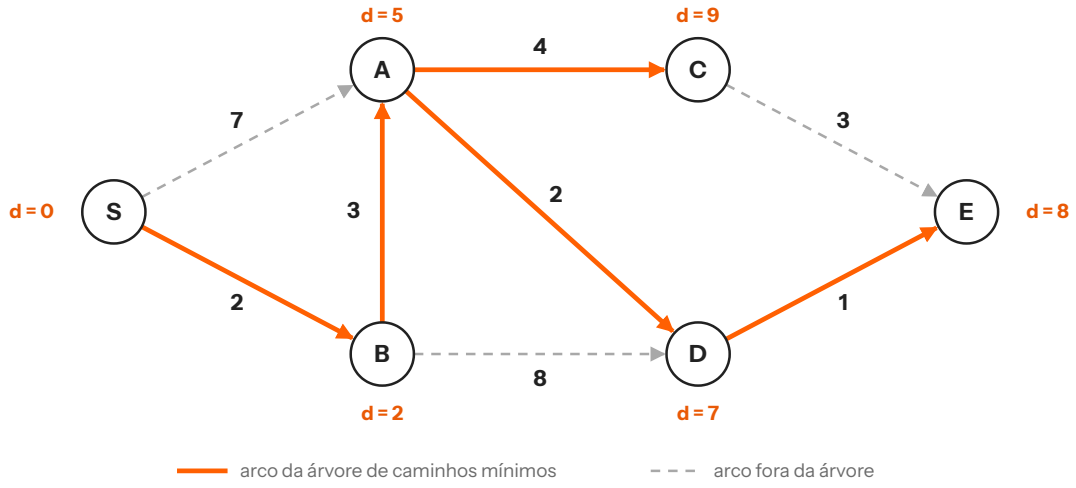


Figura 5.1 · Árvore de caminhos mínimos a partir de S; o caminho até E custa 8. Os arcos tracejados não melhoram nenhuma distância final: S→A (7) perde para S→B→A (5), B→D (10) para B→A→D (7) e C→E (12) para D→E (8).

5.4 Pesos negativos e Bellman-Ford

Dijkstra pode errar com um único arco negativo. Com S→A 2, S→B 4, B→A −3 e A→C 1, ele fixa A = 2 e C = 3 antes de examinar B. Os valores corretos são A = 1 (por S→B→A) e C = 2.

Bellman-Ford relaxa todos os arcos n − 1 vezes, número suficiente porque um caminho mínimo sem ciclos tem no máximo n − 1 arcos. Uma rodada extra denuncia ciclo negativo.

```
BELLMAN-FORD(G, w, s)
  dist[s] ← 0; dist[v] ← ∞ para os demais
  repita n - 1 vezes:
      para cada arco (u, v): RELAXA(u, v)
  para cada arco (u, v):
      se dist[u] + w(u, v) < dist[v]: devolva “há ciclo negativo”
  devolva dist
```

O custo é O(n·m). Em DAGs, relaxar os arcos na ordem topológica resolve o problema em O(n + m), mesmo com pesos negativos.

5.5 Floyd-Warshall

Floyd-Warshall usa programação dinâmica: na etapa k, permite que o vértice k sirva de intermediário.

$$D^{(k)}[i][j] = \min(D^{(k-1)}[i][j],\ D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$$

O custo é $O(n^3)$ de tempo e $O(n^2)$ de memória. O algoritmo aceita arcos negativos; se algum D[i][i] ficar negativo, há ciclo negativo.

Exemplo passo a passo com os arcos 1→2 3, 1→3 8, 2→3 2, 2→4 5, 3→4 1 e 4→1 2. A matriz inicial $D^{(0)}$ traz os pesos diretos, e a final, $D^{(4)}$, as distâncias mínimas entre todos os pares:

$D^{(0)}$	1	2	3	4
1	0	3	8	∞
2	∞	0	2	5
3	∞	∞	0	1
4	2	∞	∞	0

→

$D^{(4)}$	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

As atualizações de cada etapa:

1. **k = 1:** D[4][2] = 2 + 3 = 5 e D[4][3] = 2 + 8 = 10.
2. **k = 2:** D[1][3] = 3 + 2 = 5, D[1][4] = 3 + 5 = 8 e D[4][3] = 5 + 2 = 7.
3. **k = 3:** D[1][4] = 5 + 1 = 6 e D[2][4] = 2 + 1 = 3.
4. **k = 4:** D[2][1] = 3 + 2 = 5, D[3][1] = 1 + 2 = 3 e D[3][2] = 1 + 5 = 6.

5.6 Qual algoritmo usar

SITUAÇÃO	ALGORITMO	COMPLEXIDADE
Uma origem, sem pesos	BFS	$O(n + m)$
Uma origem, pesos não negativos	Dijkstra	$O((n + m) \log n)$ com heap; $O(n^2)$ com vetor
Uma origem, pesos negativos	Bellman-Ford	$O(n \cdot m)$
Uma origem, grafo acíclico (DAG)	relaxamento em ordem topológica	$O(n + m)$
Todos os pares	Floyd-Warshall	$O(n^3)$

ATENÇÃO

- Somar uma constante a todos os pesos pode mudar o caminho mínimo, porque penaliza caminhos com mais arcos. No exemplo de Dijkstra, somando 10 a cada arco, S → A → D → E (custo 40) passa a vencer S → B → A → D → E (custo 48).
- Multiplicar todos os pesos por uma constante positiva não muda os caminhos mínimos.
- Uma aresta não direcionada de peso negativo já forma um ciclo negativo (u → v → u).
- Dijkstra pode errar com arcos negativos mesmo sem ciclo negativo.

QUESTÃO NO ESTILO ENADE

Avalie as asserções a seguir e a relação proposta entre elas.

I. O algoritmo de Dijkstra pode produzir distâncias incorretas em grafos com arcos de peso negativo, mesmo sem ciclos negativos.

PORQUE

II. Ao extrair um vértice da fila de prioridade, Dijkstra considera definitiva a sua distância, supondo que nenhum caminho examinado depois possa reduzi-la.

A respeito dessas asserções, assinale a opção correta.

(A) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.

(B) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.

(C) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.

(D) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.

(E) As asserções I e II são proposições falsas.

GABARITO COMENTADO

Gabarito: A. A asserção I é verdadeira, como mostra o contraexemplo da seção 5.4. A asserção II descreve o passo guloso do algoritmo e explica por que um arco negativo examinado depois quebra a garantia.

Exercícios

1. Execute Dijkstra a partir de A no grafo do exemplo.
2. Qual caminho Floyd-Warshall encontra de 3 para 2? Qual o custo?
3. Um grafo tem arcos negativos, e é preciso saber se existe ciclo negativo. Qual algoritmo usar e com que custo?
4. Por que Bellman-Ford precisa de n − 1 rodadas de relaxamento?

CAPÍTULO 6

6 GRAFOS EULERIANOS E HAMILTONIANOS

A IDEIA CENTRAL DO CAPÍTULO

Um grafo conexo tem circuito euleriano, que passa uma vez por cada aresta, se e somente se todos os vértices têm grau par, condição testada em O(n + m). Já o ciclo hamiltoniano, que passa uma vez por cada vértice, não tem caracterização simples: decidir se ele existe é NP-completo.

6.1 Grafos eulerianos

- **Trilha euleriana:** percorre cada aresta exatamente uma vez. Quando fechada, chama-se circuito (ou ciclo) euleriano.
- **Teorema de Euler:** um grafo conexo, ignorando vértices isolados, tem circuito euleriano se e somente se todo vértice tem grau par.
- Há trilha euleriana aberta se e somente se existem exatamente dois vértices de grau ímpar; a trilha começa em um deles e termina no outro.
- Em digrafos, há circuito euleriano quando o grafo subjacente é conexo e $d^+(v) = d^-(v)$ para todo v.
- Königsberg tem quatro vértices de grau ímpar (5, 3, 3, 3), então não admite trilha nem circuito.

Dois algoritmos constroem o percurso:

- **Fleury:** parte de um vértice (de grau ímpar, se houver), apaga cada aresta atravessada e só usa uma ponte do grafo restante quando não há alternativa. Custo $O(m^2)$.
- **Hierholzer:** monta um circuito qualquer e depois encaixa nele sub-circuitos que partem de vértices com arestas livres. Custo O(n + m).

6.2 Exemplo passo a passo: a casinha

A casinha é o quadrado ABCD com as diagonais AC e BD e o telhado CE e ED, num total de 8 arestas. Os graus são A 3, B 3, C 4, D 4 e E 2. Com exatamente dois vértices ímpares, A e B, há trilha euleriana de A até B, mas não circuito.

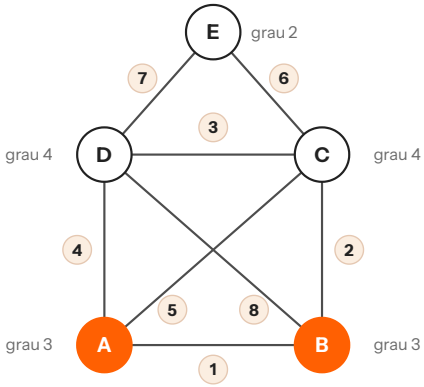


Figura 6.1 · A casinha: só A e B têm grau ímpar, então a trilha euleriana começa em A e termina em B. Os números nas arestas dão a ordem do percurso de Fleury; a diagonal DB, evitada no passo 4 por ser ponte, só é percorrida no fim.

Fleury a partir de A:

PASSO	VÉRTICE ATUAL	ARESTAS LIVRES NELE	ESCOLHA E MOTIVO
1	A	AB, AC, AD	AB; nenhuma é ponte
2	B	BC, BD	BC; não é ponte
3	C	CA, CD, CE	CD; não é ponte
4	D	DA, DB, DE	DA; DB virou ponte e levaria a B sem volta
5	A	AC	AC, a única
6	C	CE	CE, a única
7	E	ED	ED, a única
8	D	DB	DB, a única; a trilha termina em B

A trilha é A–B–C–D–A–C–E–D–B. O passo 4 é o que importa: ir de D para B encerraria o percurso cedo, deixando AC, CE e ED sem uso.

6.3 Problema do carteiro chinês

O carteiro precisa de uma rota fechada de custo mínimo que passe por todas as arestas pelo menos uma vez. Se o grafo é euleriano, a resposta é o próprio circuito euleriano, de custo igual à soma dos pesos.

Se não for, emparelham-se os vértices de grau ímpar pelo menor custo total de caminhos mínimos, duplicam-se esses caminhos e procura-se o circuito; há solução polinomial (Edmonds e Johnson). Na casinha com arestas de peso 1, o caminho mínimo entre A e B é a própria aresta AB: duplicada, ela torna todos os graus pares, e a rota custa 8 + 1 = 9.

6.4 Grafos hamiltonianos

- **Ciclo hamiltoniano:** passa por cada vértice exatamente uma vez e volta ao início. **Caminho hamiltoniano:** o mesmo, sem voltar.
- Decidir se existe ciclo hamiltoniano é NP-completo (Karp, 1972).

- Condições suficientes, para $n \geq 3$:
  - **Dirac:** se $\delta(G) \geq n/2$, G é hamiltoniano.
  - **Ore:** se $d(u) + d(v) \geq n$ para todo par u, v não adjacente, G é hamiltoniano.
- **Condições que impedem:** vértice de grau 1, vértice de corte, ou bipartido com partes de tamanhos diferentes, como $K_{2,3}$. Em geral, se remover k vértices deixa mais de k componentes, não há ciclo hamiltoniano.
- **Exemplos:** K_n, com $n \geq 3$, é hamiltoniano. C_n também é, embora $\delta = 2$ não satisfaça Dirac. O grafo de Petersen não é. A casinha é: A–B–C–E–D–A.

6.5 Problema do caixeiro viajante (PCV)

O PCV procura o ciclo hamiltoniano de custo mínimo em um grafo completo ponderado. Em K_n com custos simétricos, há $(n - 1)!/2$ ciclos distintos, o que torna inviável testar todos:

CIDADES (N)	CICLOS DISTINTOS, (N − 1)!/2
4	3
5	12
10	181.440
15	43.589.145.600
20	cerca de $6{,}1 \times 10^{16}$

As abordagens se dividem em três grupos:

- **Heurísticas,** como vizinho mais próximo e 2-opt: rápidas, mas sem garantia de ótimo.
- **Aproximações,** quando vale a desigualdade triangular: a heurística baseada na AGM custa no máximo 2 vezes o ótimo, e a de Christofides, no máximo 1,5 vez.
- **Métodos exatos,** como a programação dinâmica de Held-Karp, em $O(n^2 2^n)$, e o branch-and-bound.

6.6 Euler × Hamilton e a fronteira entre P e NP

Problemas parecidos podem ter complexidades muito diferentes, e o ENADE explora esse contraste.

PROBLEMA (VERSÃO DE DECISÃO)	CLASSE	COMO SE RESOLVE
Circuito euleriano	P	teste de graus e conexidade, $O(n + m)$
Carteiro chinês, grafo não direcionado	P	caminhos mínimos e emparelhamento
Árvore geradora mínima	P	Kruskal, Prim
Caminho mínimo sem ciclo negativo	P	Dijkstra, Bellman-Ford
Fluxo máximo	P	Ford-Fulkerson (Edmonds-Karp)
2-coloração (bipartição)	P	BFS
Ciclo hamiltoniano	NP-completo	busca exaustiva com poda
Caixeiro viajante	NP-completo	heurísticas ou métodos exatos exponenciais
k-coloração, $k \geq 3$	NP-completo	heurísticas, como o guloso
Clique, conjunto independente, cobertura de vértices	NP-completos	métodos exponenciais; a cobertura tem aproximação de fator 2

ATENÇÃO

- Euler trata de arestas; Hamilton, de vértices.
- Dirac e Ore são condições suficientes: falhar nelas não prova que o grafo não é hamiltoniano.
- Grafo com arestas em duas componentes diferentes não é euleriano, mesmo que todos os graus sejam pares.
- K_n é euleriano só para n ímpar, quando o grau n − 1 é par.

QUESTÃO NO ESTILO ENADE

Uma prefeitura planeja duas rotas. Na rota 1, um caminhão de varrição deve passar por todas as ruas de um bairro e voltar à garagem. Na rota 2, um técnico deve visitar uma vez cada um dos 30 semáforos e voltar à base, percorrendo a menor distância. Modelando o bairro como grafo, com esquinas como vértices e ruas como arestas, é correto afirmar:

(A) As duas rotas são ciclos hamiltonianos, e o problema da rota 1 é NP-completo.

(B) A rota 1 é um circuito euleriano quando todas as esquinas têm grau par, e a rota 2 é uma instância do caixeiro viajante.

(C) A rota 1 existe se e somente se o grafo é completo, e a rota 2 é obtida com Dijkstra.

(D) A rota 2 é um circuito euleriano, e a rota 1 é resolvida pelo algoritmo de Kruskal.

(E) As duas rotas são obtidas em tempo polinomial por busca em largura.

GABARITO COMENTADO

Gabarito: B.

- A rota 1 cobre arestas (ruas): é um problema euleriano, ou do carteiro chinês se houver esquinas de grau ímpar.
- A rota 2 cobre pontos com custo mínimo: é o caixeiro viajante, NP-difícil.
- A erra a classificação da rota 1; C inventa a exigência de grafo completo; D inverte os conceitos; E ignora que o PCV não tem algoritmo polinomial conhecido.

Exercícios

1. Para quais valores de n o grafo K_n tem circuito euleriano?
2. Para quais r e s o grafo $K_{r,s}$ tem circuito euleriano? E ciclo hamiltoniano?
3. O grafo de Petersen, 3-regular com 10 vértices, é euleriano?
4. Use o teorema de Dirac para mostrar que $K_{3,3}$ é hamiltoniano e exiba um ciclo.
5. Quantos ciclos hamiltonianos distintos tem K_6?

CAPÍTULO 7

7 PLANARIDADE

A IDEIA CENTRAL DO CAPÍTULO

Um grafo é planar quando pode ser desenhado no plano sem cruzamento de arestas. Pela fórmula de Euler, todo grafo planar conexo satisfaz $n - m + f = 2$, o que limita um grafo planar simples a $m \leq 3n - 6$ arestas.

7.1 Definições

- **Grafo planar:** admite pelo menos um desenho sem cruzamentos. **Grafo plano:** um desses desenhos, já fixado.
- **Face** (ou região): cada parte em que o desenho divide o plano, incluindo a face externa, ilimitada.
- **Grau de uma face:** número de arestas na sua fronteira, e uma ponte conta duas vezes. A soma dos graus das faces é $2m$.

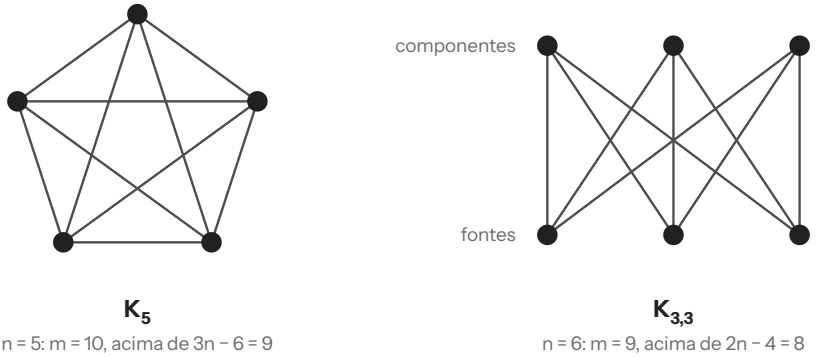
7.2 Fórmula de Euler e consequências

$$n - m + f = 2$$

A fórmula vale para grafos planares conexos; com k componentes, $n - m + f = k + 1$. Para grafos planares simples e conexos com $n \geq 3$, ela gera três consequências:

1. Toda face tem grau pelo menos 3, então $2m \geq 3f$. Substituindo f na fórmula, obtém-se $m \leq 3n - 6$.
2. Sem triângulos, como nos bipartidos, toda face tem grau pelo menos 4, e o limite cai para $m \leq 2n - 4$.
3. Todo grafo planar simples tem um vértice de grau no máximo 5.

7.3 K_5, $K_{3,3}$ e o teorema de Kuratowski



K_5
n = 5: m = 10, acima de 3n − 6 = 9

$K_{3,3}$
n = 6: m = 9, acima de 2n − 4 = 8

Figura 7.1 · K_5 e $K_{3,3}$, os dois grafos do teorema de Kuratowski, ultrapassam o limite de arestas de Euler. $K_{3,3}$ é o problema da placa de circuito da questão deste capítulo: três componentes ligados a três fontes. Qualquer desenho de K_5 ou de $K_{3,3}$ no plano tem pelo menos um cruzamento.

- K_5 tem $n = 5$ e $m = 10 > 3 \cdot 5 - 6 = 9$, então não é planar.
- $K_{3,3}$ é bipartido, com $n = 6$ e $m = 9 > 2 \cdot 6 - 4 = 8$, então não é planar.
- **Teorema de Kuratowski (1930):** G é planar se e somente se não contém subdivisão de K_5 nem de $K_{3,3}$. Uma subdivisão troca arestas por caminhos.
- Os limites de arestas são necessários, não suficientes. $K_{3,3}$ respeita $m \le 3n - 6$ ($9 \le 12$) e não é planar; o grafo de Petersen ($n = 10$, $m = 15 \le 24$) também não é.

7.4 Exemplos passo a passo

Problema 1: um grafo planar conexo tem 10 vértices, todos de grau 3. Quantas faces tem qualquer desenho plano dele?

1. Pelo aperto de mãos, $2m = 10 \cdot 3 = 30$, então $m = 15$.
2. Pela fórmula de Euler, $f = 2 - n + m = 2 - 10 + 15 = 7$.

Problema 2: a casinha do Capítulo 6 ($n = 5$, $m = 8$) é planar?

1. **Teste de arestas:** $8 \le 3 \cdot 5 - 6 = 9$, então o teste não exclui a planaridade.
2. **Desenho:** leve a diagonal BD para fora do quadrado, contornando o vértice A por baixo e pela esquerda; nenhuma aresta se cruza.
3. **Conferência:** $f = 2 - 5 + 8 = 5$ faces, que são os triângulos ABC, ACD, CDE e ABD e a face externa.

7.5 Grafo dual e mapas

Dado um grafo plano G, o dual G* tem um vértice para cada face de G e uma aresta para cada aresta de G, ligando as duas faces que ela separa. Colorir as regiões de um mapa equivale a colorir os vértices do dual, o que liga este capítulo ao teorema das quatro cores (Capítulo 8).

ATENÇÃO

- Um desenho com cruzamentos não prova que o grafo não é planar; outro desenho pode eliminá-los.
- A face externa entra na contagem de f.
- As desigualdades exigem grafo simples com $n \geq 3$, e $m \leq 2n - 4$ só vale sem triângulos.
- Para provar que um grafo é planar, exiba um desenho. Para provar que não é, use as desigualdades ou encontre uma subdivisão de K_5 ou $K_{3,3}$.

QUESTÃO NO ESTILO ENADE

Um projetista de placas de circuito precisa ligar cada um de três componentes, A, B e C, a cada uma de três fontes, X, Y e Z, em uma única camada e sem cruzamento de trilhas. É correto afirmar:

(A) É possível, pois $K_{3,3}$ tem 9 arestas, menos que $3n - 6 = 12$.

(B) É impossível, pois $K_{3,3}$ é bipartido, não tem triângulos e precisaria ter no máximo $2n - 4 = 8$ arestas.

(C) É possível, desde que as trilhas sejam curvas.

(D) É impossível, pois $K_{3,3}$ contém K_5 como subgrafo.

(E) É possível, pois todo grafo bipartido é planar.

GABARITO COMENTADO

Gabarito: B. O problema é desenhar $K_{3,3}$ sem cruzamentos.

- A confunde condição necessária com suficiente.
- C: curvar as trilhas não muda a planaridade.
- D: $K_{3,3}$ é bipartido e não contém triângulo, muito menos K_5.
- E: o próprio $K_{3,3}$ é um bipartido não planar.

Exercícios

1. Um grafo planar conexo tem 6 vértices, todos de grau 4. Quantas faces ele tem?
2. Quantas faces tem K_4 desenhado no plano?
3. Qual é o número máximo de arestas de um grafo planar simples com 12 vértices?
4. Um grafo bipartido, planar e conexo, com 10 vértices, pode ter 17 arestas?
5. O cubo Q_3 é planar? Quantas faces tem?

CAPÍTULO 8

8 COLORAÇÃO DE GRAFOS

A IDEIA CENTRAL DO CAPÍTULO

O número cromático $\chi(G)$ é o menor número de cores que permite colorir os vértices sem que vizinhos tenham a mesma cor. Todo grafo planar tem $\chi \leq 4$, mas decidir se $\chi(G) \leq 3$ é NP-completo; na prática, usam-se heurísticas como o algoritmo guloso.

8.1 Definições e valores conhecidos

- **Coloração própria:** vértices adjacentes recebem cores diferentes. Cada classe de cor é um conjunto independente, então colorir é particionar V em conjuntos independentes.
- **Limites:** $\omega(G) \leq \chi(G) \leq \Delta(G) + 1$, em que $\omega(G)$ é o tamanho da maior clique.
- **Teorema de Brooks:** se G é conexo e não é completo nem ciclo ímpar, $\chi(G) \leq \Delta(G)$.

GRAFO	χ
K_n	n
C_n com n par	2
C_n com n ímpar	3
Árvore com n ≥ 2	2
Bipartido com pelo menos uma aresta	2
Planar	no máximo 4
Petersen	3

8.2 Algoritmo guloso e Welsh-Powell

O guloso percorre os vértices numa ordem fixa e dá a cada um a menor cor ainda não usada pelos vizinhos já coloridos.

```
GULOSO(G, ordem v1, ..., vn)
  para i de 1 até n:
      cor[vi] ← menor cor que nenhum vizinho já colorido de vi usa
```

Ele usa no máximo $\Delta + 1$ cores, mas o resultado depende da ordem. Welsh-Powell ordena os vértices por grau decrescente e preenche uma cor de cada vez, percorrendo a lista; continua sendo heurística e não garante $\chi(G)$.

8.3 Exemplo passo a passo: calendário de provas

Seis disciplinas têm prova final: Grafos (GRA), Banco de Dados (BD), Redes (RED), Inteligência Artificial (IA), Compiladores (COMP) e Engenharia de Software (ES). Uma aresta liga duas disciplinas com algum aluno em comum, e cada cor é um horário; a coloração própria garante que ninguém tem duas provas no mesmo horário.

As arestas são GRA–BD, GRA–RED, BD–RED, GRA–IA, RED–IA, IA–COMP, BD–COMP, COMP–ES e ES–IA. Os graus são IA 4; GRA, BD, RED e COMP 3; ES 2. A ordem de Welsh-Powell, com empates em ordem alfabética, é IA, BD, COMP, GRA, RED, ES.

HORÁRIO	PERCURSO NA LISTA	DISCIPLINAS
1	IA recebe a cor; BD não é vizinha de IA; as demais são vizinhas de IA	IA, BD
2	COMP recebe a cor; GRA não é vizinha de COMP; RED é vizinha de GRA; ES é vizinha de COMP	COMP, GRA
3	RED recebe a cor; ES não é vizinha de RED	RED, ES

Três horários bastam, e menos não é possível: GRA, BD e RED formam um triângulo, então $\chi \geq \omega = 3$.

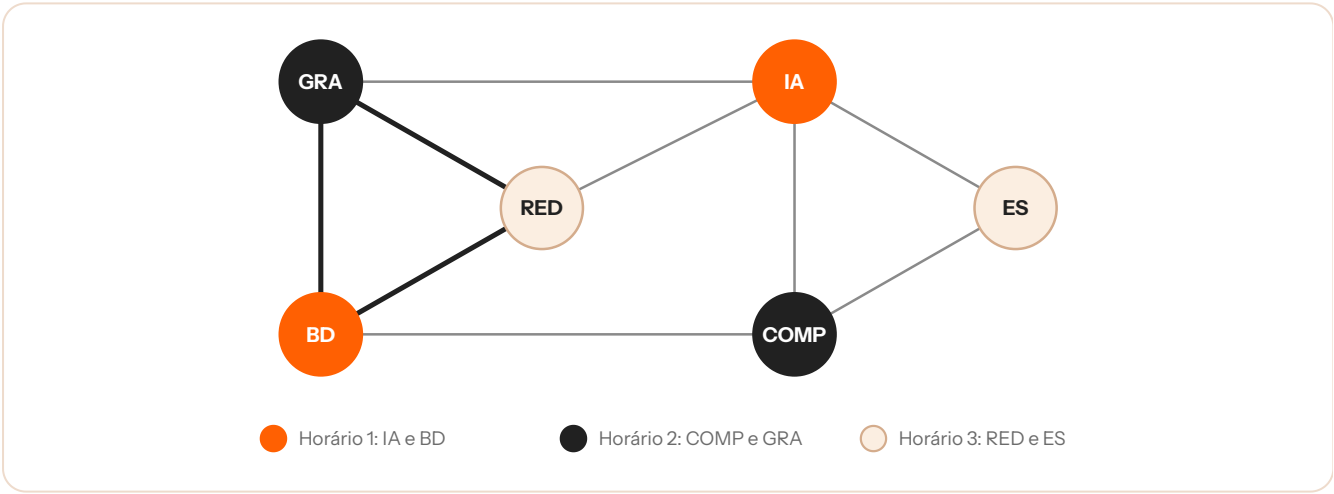


Figura 8.1 · Grafo de conflitos das provas, colorido por horário. Disciplinas da mesma cor nunca são vizinhas, então podem ter prova no mesmo horário; as arestas mais grossas formam o triângulo GRA–BD–RED, que obriga a usar três cores.

8.4 Coloração de arestas e de mapas

- **Índice cromático $\chi'(G)$:** menor número de cores para as arestas, sem duas arestas da mesma cor num mesmo vértice. Pelo teorema de Vizing, $\Delta \leq \chi'(G) \leq \Delta + 1$, e em bipartidos $\chi' = \Delta$ (König). Aplicação: rodadas de um torneio, em que cada cor é uma rodada.
- **Mapas:** as regiões viram vértices do dual (Capítulo 7). O teorema das quatro cores (Appel e Haken, 1976, com prova assistida por computador) garante $\chi \leq 4$ para todo grafo planar; o de cinco cores tem prova curta (Heawood, 1890).

ATENÇÃO

- O guloso pode errar muito: em $K_{n,n}$ sem um emparelhamento perfeito, que é bipartido, uma ordem ruim gasta n cores, embora 2 bastem.
- $\chi(G) \geq \omega(G)$, mas pode ser maior: C_5 tem $\omega = 2$ e $\chi = 3$.
- 2-coloração se decide em $O(n + m)$ com BFS; 3-coloração já é NP-completa.
- Quatro cores bastam para grafos planares, não para qualquer grafo: K_5 precisa de 5.

QUESTÃO NO ESTILO ENADE

Uma universidade deve marcar as provas finais de modo que nenhum estudante tenha duas provas no mesmo horário. Ela modela o problema com um grafo de conflitos: disciplinas são vértices, e uma aresta liga disciplinas com algum aluno em comum. Considere as afirmações:

I. O número mínimo de horários é o número cromático do grafo de conflitos.

II. Se o grafo de conflitos contém K_4 como subgrafo, são necessários pelo menos 4 horários.

III. O algoritmo guloso sempre encontra o número mínimo de horários quando os vértices são ordenados por grau decrescente.

IV. Se o grafo de conflitos for bipartido, dois horários bastam.

É correto apenas o que se afirma em:

(A) I e III.

(B) II e IV.

(C) I, II e IV.

(D) I, III e IV.

(E) II, III e IV.

GABARITO COMENTADO

Gabarito: C.

- I é verdadeira: cada horário é uma cor, e a coloração própria evita conflitos.
- II é verdadeira: os quatro vértices de K_4 são dois a dois vizinhos.
- III é falsa: Welsh-Powell é heurística e não garante o ótimo.
- IV é verdadeira: as duas partes da bipartição servem de horários.

Exercícios

1. Qual é $\chi(C_7)$? E $\chi(K_{4,5})$?
2. Uma roda é formada por C_5 mais um vértice central ligado a todos os outros. Qual é seu número cromático?
3. Aplique o guloso ao grafo do exemplo na ordem ES, IA, COMP, RED, GRA, BD.
4. Qual é o índice cromático de $K_{3,3}$? Interprete como um torneio entre dois times de três jogadores.
5. Por que um grafo com triângulo nunca é 2-colorível?

CAPÍTULO 9

9 FLUXOS EM REDES E EMPARELHAMENTOS

A IDEIA CENTRAL DO CAPÍTULO

O valor do fluxo máximo de s a t é igual à capacidade do corte mínimo, pelo teorema de Ford-Fulkerson. O mesmo raciocínio de caminhos aumentantes resolve emparelhamentos: em grafos bipartidos, o emparelhamento máximo é um fluxo máximo com capacidades 1.

9.1 Redes de fluxo

Uma rede é um digrafo com capacidade $c(u, v) \geq 0$ em cada arco, uma fonte s e um sorvedouro t. Um fluxo f obedece a duas regras:

- **Capacidade:** $0 \leq f(u, v) \leq c(u, v)$ em todo arco.
- **Conservação:** em todo vértice diferente de s e t, o que entra é igual ao que sai.

O valor do fluxo é o total que sai de s, igual ao que chega a t.

9.2 Rede residual e caminho aumentante

Na rede residual, cada arco com folga $c - f$ aparece no sentido original, e cada arco com fluxo $f > 0$ gera um arco de volta com capacidade f, que permite desfazer escolhas. Um caminho aumentante é um caminho de s a t na rede residual, e seu gargalo é a menor capacidade residual ao longo dele.

```
FORD-FULKERSON(G, c, s, t)
  f ← 0 em todos os arcos
  enquanto existir caminho aumentante P de s a t na rede residual:
      g ← gargalo de P
      some g nos arcos de P no sentido original; subtraia g nos arcos de volta
  devolva f
```

Com capacidades inteiras, o algoritmo termina em $O(m \cdot F)$, em que F é o valor do fluxo máximo. Edmonds-Karp escolhe sempre o caminho aumentante com menos arcos, por BFS, e roda em $O(n \cdot m^2)$.

9.3 Teorema do fluxo máximo e corte mínimo

Um corte (S, T) separa s, que fica em S, de t, que fica em T. Sua capacidade é a soma das capacidades dos arcos que vão de S para T; arcos de T para S não contam.

Todo fluxo vale no máximo a capacidade de qualquer corte, e no ótimo vale a igualdade: fluxo máximo = corte mínimo. Quando o algoritmo para, S é o conjunto dos vértices alcançáveis a partir de s na rede residual.

9.4 Exemplo passo a passo: Edmonds-Karp

Os arcos, com capacidades, são s→a 10, s→c 10, a→b 4, a→c 2, a→d 8, c→d 9, d→b 6, b→t 10 e d→t 10. A BFS examina os vizinhos em ordem alfabética.

ITERAÇÃO	CAMINHO AUMENTANTE (BFS)	GARGALO	FLUXO TOTAL
1	s → a → b → t	4	4
2	s → a → d → t	6	10
3	s → c → d → t	4	14
4	s → c → d → b → t	5	19

Na iteração 5, a BFS a partir de s alcança apenas c, pois s→a e c→d estão saturados; o algoritmo para com fluxo 19. O corte mínimo é S = {s, c}, com os arcos s→a (10) e c→d (9) saindo de S: 10 + 9 = 19. O arco a→c entra em S e não conta.

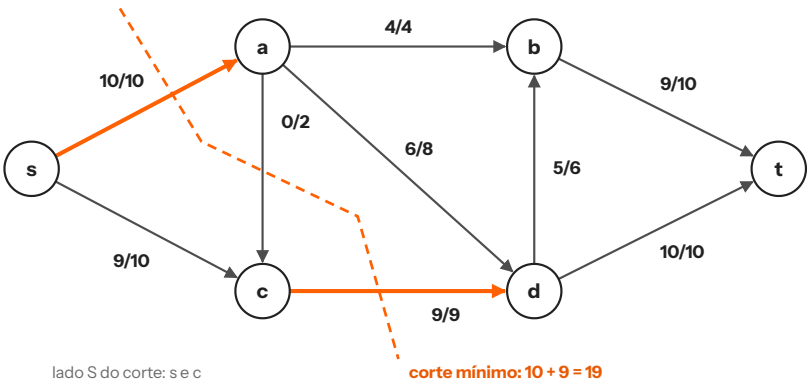


Figura 9.1 · Rede do exemplo com o fluxo final em cada arco (fluxo / capacidade) e o corte mínimo S = {s, c}. Os dois arcos destacados saem de S e estão saturados; o arco a→c volta para S com fluxo 0 e não entra na conta.

9.5 Emparelhamentos

- **Emparelhamento** (ou acoplamento) M: conjunto de arestas sem extremidades em comum. Um vértice tocado por M está saturado.
- **Maximal:** não cabe mais nenhuma aresta. **Máximo:** tem o maior tamanho possível. **Perfeito:** satura todos os vértices.
- **Caminho aumentante:** caminho alternado, com arestas fora e dentro de M, que começa e termina em vértices livres. Teorema de Berge: M é máximo se e somente se não existe caminho aumentante.
- **Teorema de Hall:** em um bipartido com partes X e Y, existe emparelhamento que satura X se e somente se todo subconjunto S de X tem pelo menos tantos vizinhos quanto elementos.
- **Teorema de König:** em bipartidos, o tamanho do emparelhamento máximo é igual ao da cobertura mínima de vértices.

Algoritmos: redução a fluxo máximo (arcos s → X, X → Y e Y → t, todos com capacidade 1); Hopcroft-Karp, em $O(m\sqrt{n})$; método húngaro, para atribuição com custos, em $O(n^3)$; e o algoritmo de Edmonds (blossom), para grafos gerais.

9.6 Exemplo passo a passo: atribuição de monitores

Ana pode monitorar Grafos ou IA; Bruno, só Grafos; Carla, IA ou Redes.

1. **Guloso:** Ana–Grafos e Carla–IA. Bruno fica sem disciplina; o emparelhamento é maximal, mas não máximo.
2. **Caminho aumentante a partir de Bruno:** Bruno–Grafos (fora de M), Grafos–Ana (em M), Ana–IA (fora), IA–Carla (em M), Carla–Redes (fora), terminando em Redes, que está livre.
3. **Troca:** as arestas fora de M entram, e as de M saem. O novo M é {Bruno–Grafos, Ana–IA, Carla–Redes}, perfeito.

ATENÇÃO

- Os arcos de volta da rede residual são essenciais; sem eles, um caminho mal escolhido trava o fluxo abaixo do máximo.
- Na capacidade do corte, só contam os arcos de S para T.
- Maximal não é o mesmo que máximo.
- Com capacidades inteiras, existe fluxo máximo inteiro, e por isso a redução de emparelhamento a fluxo funciona.

QUESTÃO NO ESTILO ENADE

Ao final do algoritmo de Ford-Fulkerson em uma rede, o conjunto dos vértices alcançáveis a partir de s na rede residual é S = {s, x, y}. Os arcos que saem de S têm capacidades 7, 5 e 3, e há um arco de capacidade 4 que entra em S. O valor do fluxo máximo é:

(A) 4. **(B)** 11. **(C)** 15. **(D)** 19.

(E) impossível de determinar sem conhecer os demais arcos.

GABARITO COMENTADO

Gabarito: C. S define um corte mínimo, e sua capacidade é a soma dos arcos que saem de S: 7 + 5 + 3 = 15.

- O arco que entra em S não conta e, no ótimo, tem fluxo zero.
- B e D subtraem ou somam esse arco (15 − 4 = 11 e 15 + 4 = 19); A considera só ele.
- E é falsa: o teorema do fluxo máximo e corte mínimo determina o valor.

Exercícios

1. No exemplo de fluxo, quais arcos terminam saturados?
2. Quantos emparelhamentos perfeitos tem $K_{n,n}$?
3. Qual é o tamanho do emparelhamento máximo em P_7 (caminho com 7 vértices)? E em C_7?
4. Um bipartido tem X = {x1, x2, x3}, e x1 e x2 só são vizinhos de y1. Existe emparelhamento que satura X?

APÊNDICE A

A QUADRO-RESUMO DE ALGORITMOS

A tabela reúne os algoritmos do ebook, o problema que cada um resolve e sua complexidade usual, com n vértices, m arestas e lista de adjacência, salvo indicação.

ALGORITMO	PROBLEMA	ESTRATÉGIA	COMPLEXIDADE	RESTRIÇÃO OU OBSERVAÇÃO
BFS	percurso; menor caminho sem pesos	fila	$O(n + m)$	distâncias valem só com pesos iguais
DFS	percurso; ciclos; componentes	pilha ou recursão	$O(n + m)$	base da ordenação topológica e de Kosaraju
Kahn	ordenação topológica	retirada de vértices com grau de entrada 0	$O(n + m)$	só em DAG
Kosaraju	componentes fortemente conexas	duas DFS, em G e em G^T	$O(n + m)$	digrafos
Kruskal	árvore geradora mínima	guloso com union-find	$O(m \log m)$	aceita pesos negativos
Prim	árvore geradora mínima	guloso com fila de prioridade	$O(m \log n)$ com heap; $O(n^2)$ com vetor	grafo conexo
Dijkstra	caminhos mínimos de uma origem	guloso com fila de prioridade	$O((n + m) \log n)$ com heap; $O(n^2)$ com vetor	pesos não negativos
Bellman-Ford	caminhos mínimos de uma origem	n − 1 rodadas de relaxamento	$O(n \cdot m)$	detecta ciclo negativo
Floyd-Warshall	caminhos mínimos entre todos os pares	programação dinâmica	$O(n^3)$	sem ciclo negativo
Fleury	trilha ou circuito euleriano	evitar pontes	$O(m^2)$	0 ou 2 vértices de grau ímpar
Hierholzer	trilha ou circuito euleriano	junção de ciclos	$O(n + m)$	0 ou 2 vértices de grau ímpar
Welsh-Powell	coloração de vértices	guloso por grau decrescente	$O(n^2)$	heurística: não garante $\chi(G)$
Edmonds-Karp	fluxo máximo	caminhos aumentantes por BFS	$O(n \cdot m^2)$	capacidades não negativas
Hopcroft-Karp	emparelhamento máximo	caminhos aumentantes em fases	$O(m\sqrt{n})$	grafo bipartido
Held-Karp	caixeiro viajante exato	programação dinâmica	$O(n^2 \cdot 2^n)$	tempo exponencial

APÊNDICE B

B RESPOSTAS DOS EXERCÍCIOS

Cada resposta traz o resultado e o raciocínio que leva a ele. Em perguntas com mais de uma solução, como ordens topológicas, outras respostas que respeitem as mesmas regras também estão corretas.

Capítulo 1 · Conceitos fundamentais

1. $10 \cdot 9/2 = 45$.
2. $2m = 24 = 3n$, logo $n = 8$.
3. Não. A soma dos graus seria $9 \cdot 3 = 27$, ímpar, o que contradiz o lema do aperto de mãos.
4. No complemento, cada vértice liga-se aos dois que não eram seus vizinhos; o resultado é o pentagrama, outro ciclo de 5 vértices.
5. Sim: $(8 - 1)(8 - 2)/2 = 21$, e $22 > 21$.

Capítulo 2 · Representação computacional de grafos

1. Linhas: 1 → (0, 1, 0, 0); 2 → (0, 0, 1, 0); 3 → (1, 0, 0, 1); 4 → (0, 0, 0, 0). O vértice 3 tem grau de saída 2 e grau de entrada 1.
2. Cada aresta gera dois 1s, logo $m = 10$.
3. Dois: 3–2–4 e 3–5–4.
4. Matriz: o grafo é denso ($m = 499.500$, cerca de $n^2/2$), e a matriz dá teste de adjacência em O(1) sem desperdício relevante de memória.

Capítulo 3 · Buscas em grafos e conexidade

1. Ordem 4, 2, 5, 1, 3. Distâncias: dist(4) = 0; dist(2) = dist(5) = 1; dist(1) = dist(3) = 2.
2. Ordem de descoberta 5, 3, 1, 2, 4, com d/f: 5 (1/10), 3 (2/9), 1 (3/8), 2 (4/7), 4 (5/6). Árvore: 53, 31, 12 e 24. Retorno: 23 e 45.
3. Para cada vértice retirado da fila, é preciso varrer uma linha inteira da matriz, com n posições.
4. Os términos são Banco de Dados 2, IA 6, Grafos 7, Estruturas de Dados 8, Matemática Discreta 10 e Programação 12. Em ordem decrescente: Programação, Matemática Discreta, Estruturas de Dados, Grafos, IA, Banco de Dados.

Capítulo 4 · Árvores e árvores geradoras mínimas

1. $15 - 11 = 4$ árvores.
2. EF 9, DE 3, CD 6, AC 2 e AB 4; é a mesma AGM, de custo 24.
3. $4^{4-2} = 16$.
4. Kruskal aceita AC, DE e AB, rejeita BC, aceita BD (7), rejeita CE e aceita EF. Nova AGM: {AC, DE, AB, BD, EF}, de custo 25.
5. Com k folhas, $n = 5 + k$, e a soma dos graus é $13 + k = 2(n - 1) = 8 + 2k$. Logo, $k = 5$.

Capítulo 5 · Caminhos mínimos

1. dist(A) = 0, dist(D) = 2, dist(E) = 3 e dist(C) = 4; S e B são inalcançáveis (∞).
2. 3 → 4 → 1 → 2, de custo 1 + 2 + 3 = 6.
3. Bellman-Ford, com a rodada extra de verificação, em O(n·m).
4. Um caminho mínimo sem ciclos tem no máximo n − 1 arcos, e cada rodada fixa corretamente pelo menos mais um arco desse caminho.

Capítulo 6 · Grafos eulerianos e hamiltonianos

1. n ímpar, pois cada vértice tem grau n − 1.
2. Euleriano: r e s pares. Hamiltoniano: r = s ≥ 2.
3. Não: todos os vértices têm grau 3, ímpar.
4. n = 6 e δ = 3 = n/2. Com partes {a, b, c} e {x, y, z}, um ciclo é a–x–b–y–c–z–a.
5. (6 − 1)!/2 = 60.

Capítulo 7 · Planaridade

1. m = 12 e f = 2 − 6 + 12 = 8; é o octaedro.
2. f = 2 − 4 + 6 = 4.
3. 3 · 12 − 6 = 30.
4. Não: o limite é 2 · 10 − 4 = 16.
5. Sim; n = 8, m = 12 e f = 2 − 8 + 12 = 6.

Capítulo 8 · Coloração de grafos

1. $\chi(C_7) = 3$ e $\chi(K_{4,5}) = 2$.
2. 4: o ciclo ímpar pede 3 cores, e o centro, vizinho de todos, pede a quarta.
3. ES 1, IA 2, COMP 3, RED 1, GRA 3, BD 2: três cores.
4. 3, pois $\chi' = \Delta$ em bipartidos: em três rodadas, cada jogador enfrenta os três adversários.
5. Os três vértices do triângulo são dois a dois vizinhos e precisam de três cores distintas.

Capítulo 9 · Fluxos em redes e emparelhamentos

1. s→a (10 de 10), a→b (4 de 4), c→d (9 de 9) e d→t (10 de 10).
2. n!, um para cada permutação.
3. 3 nos dois casos.
4. Não. Pelo teorema de Hall, S = {x1, x2} tem um só vizinho, menos que seus 2 elementos.

APÊNDICE C

C REFERÊNCIAS

BOAVENTURA NETTO, P. O. **Grafos**: teoria, modelos, algoritmos. 4. ed. São Paulo: Blucher, 2006.

CORMEN, T. H. et al. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

GOLDBARG, M.; GOLDBARG, E. **Grafos**: conceitos, algoritmos e aplicações. Rio de Janeiro: Elsevier, 2012.

PACÍFICO, L. **Revisão de Teoria dos Grafos**: ENADE 2026. CESAR School, 2026. Inclui os slides da disciplina. Disponível em: https://laps-wq.github.io/revisao-grafos/. Acesso em: 2 nov. 2026.

SZWARCFITER, J. L. **Teoria computacional de grafos**. Rio de Janeiro: Elsevier, 2018.